

## I312 - Bases de Datos

### Trabajo Práctico 1

De la planilla al servidor: repensando el camino del helado



Athina Salim  
Ana Paula Tissera  
Naomi Couriel  
Ana Holcman

## Resumen

Este trabajo presenta el diseño e implementación de una base de datos para una cadena de heladerías. La necesidad surge a partir de un sistema previo basado en planillas de Excel que se actualizaban e imprimían diariamente, dificultando el control de información.

El desarrollo abarcó la construcción del modelo entidad-relación, la obtención del modelo lógico normalizado y la implementación del modelo físico en PostgreSQL. Se modelaron entidades como sucursales, usuarios, productos y pedidos, incluyendo sus relaciones y restricciones. Se logró un sistema capaz de almacenar y gestionar los pedidos de manera estructurada, segura y más eficiente, incluyendo la identificación de los responsables en cada etapa del proceso de abastecimiento.

Como proyección futura se plantea la integración de esta base con una aplicación web, de forma tal que los pedidos puedan emitirse y gestionarse en línea, eliminando completamente el uso de papel y garantizando un flujo de información confiable y actualizado en tiempo real.

## 1 Introducción

Una base de datos almacena información en una unidad lógica junto a los metadatos necesarios para su procesamiento. Estos sistemas son instrumentos de gran utilidad para gestionar grandes ficheros y facilitar la consulta de información. Además, puede definirse un esquema de permisos que establece qué personas o programas pueden acceder a los datos y a cuáles, con el objetivo de presentar el contenido de forma adecuada y clara. [1]

En este trabajo se desarrolla el diseño e implementación de una base de datos para una cadena de heladerías con múltiples sucursales abastecidas desde una fábrica central. El objetivo principal es modelar el proceso mediante el cual una sucursal emite un pedido y se gestiona su entrega, representando las operaciones reales del negocio. Previo a la realización de este trabajo, la franquicia gestionaba los pedidos mediante un archivo 'Excel' que se sobrescribía diariamente para registrar uno nuevo. El historial de qué sucursal, bajo la responsabilidad de qué encargado, había emitido cada pedido se conservaba imprimiendo dichos archivos y almacenándolos en carpetas físicas, las cuales comenzaban a ocupar un espacio considerable en la oficina.

El proyecto se estructura en distintas etapas, comenzando por el desarrollo del modelo conceptual, guiado por la dinámica real del proceso de pedidos, quiénes pueden generarlos y cuáles son los distintos accesos y responsabilidades de los usuarios involucrados. Luego se construye el modelo lógico en forma de tablas normalizadas, representando las dependencias. A continuación, se implementa el modelo físico en PostgreSQL, incorporando las restricciones de integridad, claves primarias y foráneas, así como las reglas necesarias para garantizar la validez de los datos, junto con la posterior carga de información. Finalmente, se diseñan las consultas SQL para la gestión operativa y extracción de información útil.

## 2 Modelo E-R

Con el objetivo de representar el problema del abastecimiento de productos en sucursales de una misma heladería, las cuales son suministradas por una fábrica central, se diseñó el modelo entidad-relación, mostrado en la figura 1. Para construirlo se analizaron las características del negocio, las diferentes sucursales, los usuarios que participan en el sistema de aprovisionamiento hacia cada una de ellas, y los productos que se comercializan y su disponibilidad. Así, surgieron las principales entidades con sus atributos.

1. **Sucursal:** Contiene información de identificación y dirección para cada local de la heladería. Todas las sucursales son punto de venta, excepto por la Casa Central la cual es administración, punto de venta y la fábrica central.
2. **Usuario:** Concentra datos personales. Es un supertipo que se especializa en **Empleado**, **Proveedor** o **Administrador**, dependiendo el rol de la persona. Un empleado trabaja en un punto de venta; un administrador se ocupa de tareas de gestión generales como controlar la producción, administrar stock y gestionar pagos, entre otras. Por último, un proveedor se encarga de realizar el delivery de los pedidos desde fábrica hacia las distintas heladerías. Trabaja *para* la empresa (no es externo) y solo transporta productos que provienen de la fábrica. Si se requiere algún insumo adicional, la empresa lo adquiere, lo incorpora al stock de fábrica y recién entonces es enviado a la sucursal correspondiente. Los administradores y proveedores están *asociados* a la sucursal 'Casa Central', la cual, además de tener su parte de punto de venta (por ende también empleados), es

donde se encuentra la fábrica y el edificio de administración, como ya fue mencionado. Puede existir un Usuario que aún no este especificado en algún rol.

3. **Producto:** Representa la mercancía que las sucursales pueden pedir, de acuerdo con sus necesidades. Históricamente ya tenían cada uno su ID, y en este trabajo se respetan los mismos.
4. **Categoría:** Permite organizar la *variedad* de productos (por ejemplo: helados, tortas, tartas).
5. **Familia:** Especializa a la categoría helado según la *base de crema* utilizada en la producción del sabor (por ejemplo: chocolate, dulce de leche, crema blanca).
6. **Pedido:** Representa la solicitud de productos.
7. **Entrega:** Registra los pedidos recibidos.

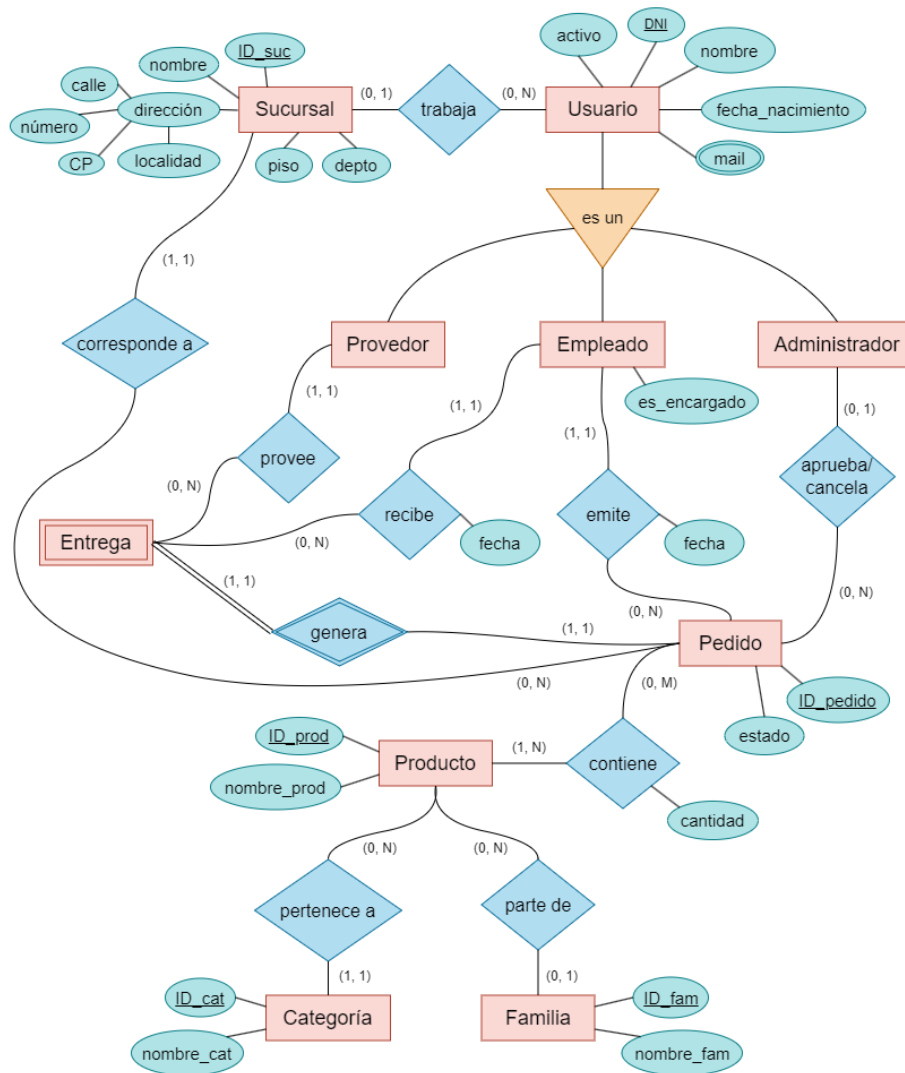


Figura 1: Modelo E-R

Luego de plantear las entidades, se profundizó en las relaciones entre ellas. La primera, y probablemente la más sencilla, es Usuario *trabaja* en Sucursal. La cardinalidad es 1-N debido a que un usuario puede trabajar solo en una sucursal, pero en una sucursal pueden trabajar varios usuarios. Se dejó abierta la posibilidad 0 en ambos lados de la relación debido a que se puede querer registrar sucursales que estén por inaugurar pero aún no lo hayan hecho, y usuarios que aún no hayan sido seleccionados para una sucursal específica pero que ya hayan sido contratados. Usuario tiene una *relación disjunta* con Proveedor, Empleado y Administrador, dado que un Usuario puede ser uno y solo uno de estos tipos. Se permite la existencia de usuarios que aún no han sido registrados como ningún tipo específico.

Para continuar, como el interés de este proyecto es modelar el proceso de los pedidos, se pensó en el ‘camino’ que recorre uno, y así fueron surgiendo las relaciones. En la figura 2 se muestra un grafo que representa los caminos posibles entre los diferentes estados que puede recorrer un pedido.

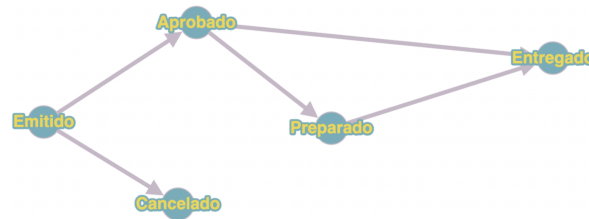


Figura 2: Grafo de Estados y Caminos que puede recorrer un Pedido

Para empezar, los Empleados encargados son quienes *emiten* los Pedidos; esta actividad es completamente individual y se basa en los criterios del encargado a la hora de ver qué es necesario reponer. Un empleado encargado puede llegar a hacer muchos pedidos diferentes, como también puede suceder que uno nunca haya hecho ninguno si empezó a trabajar recientemente. Es necesario guardar la fecha de emisión para luego poder corroborar que la entrega de este no haya demorado.

Una vez el pedido existe, desde la Casa Central un administrador debe verlo y *aprobarlo/cancelarlo*. De ser el primer caso, el pedido pasa al estado ‘aprobado’ y se admite que sea preparado. Caso contrario, el pedido pasa a estado *cancelado*. Un pedido solo puede ser cancelado por un administrador, pero una vez aprobado ya no puede ser cancelado. De ser aprobado, el pedido puede pasar al estado *preparado* (cargado en el camión para reparto) y luego a *entregado*, o, si se cumplan ciertas condiciones (como ser un ‘pedido sencillo’ compuesto por productos ya disponibles en el camión), el pedido pasa directamente de *aprobado* a *entregado*. Entonces un pedido tiene 5 diferentes estados: ‘emitido’, ‘aprobado’ o ‘cancelado’, ‘preparado’ y ‘entregado’. Los pedidos tienen la relación *Corresponde a* con la entidad Sucursal debido a que necesitamos guardar a cuál de ellas pertenece cada pedido. Es importante que salga desde Sucursal y no desde Usuario dado que, si un empleado es transferido de una sucursal a otra, necesitamos que los pedidos que hubiese realizado previo a esto se mantengan asociados a la sucursal correspondiente. Si un pedido pasa al estado *entregado*, se genera entonces una Entrega, la cual es identificada a partir del id del pedido, dado que se va a entregar exacto lo que se pidió. Una Entrega no puede existir sin un Pedido, por consiguiente es débil del mismo. Un pedido puede aún no haber sido aprobado por nadie; cuando lo sea, será debido a un único administrador. Un administrador podría no haber aprobado ningún pedido.

Los encargados de hacer las entregas son los proveedores. Así surge la relación *Provee*, cuyas cardinalidades se explican porque un Proveedor nuevo puede aún no haber hecho ninguna Entrega, mientras que uno que haya estado un tiempo en ese puesto puede haber realizado N. A una Entrega sí o sí la debe haber hecho un Proveedor. Si bien en la vida real a una misma Entrega la realizan entre varios proveedores, solamente uno es el responsable de esta, por ende, únicamente interesa su información.

Los empleados de las diferentes sucursales se encargan de *recibir* estas entregas. Otra vez, quizás más de uno ayuda a recibir la mercadería, pero es uno el responsable de fijarse que esta haya llegado correctamente, resultando en que solamente se quiere su información. Puede suceder que a un empleado nunca le haya tocado recibir mercancía.

Con estas relaciones diagramadas, faltaba responder qué contenía exactamente cada pedido, y así surge la relación *Contiene*, entre Producto y Pedido, la cual tiene como atributo a cantidad, puesto que se desea conocer cuánto de un producto exacto se está pidiendo. Un pedido como mínimo debe tener un producto. Como se mencionó previamente, un producto es una mercadería exacta, por ejemplo ‘limón’ o ‘higos con nueces’. Además de conocer el nombre del producto, interesa su categoría y su familia, entonces se modelaron también las relaciones *pertenece a* y *parte de*. Un producto puede pertenecer a una única categoría y puede o no ser parte de una familia. En el presente, la única categoría subdividida en familia es helado, pero en un futuro se busca dividir a las Tortas Heladas en las mismas familias en las que se divide hoy al helado, dado que estas los utilizan como ingredientes. De esta manera, se decidió dejar el modelo listo para poder introducir este cambio sin problemas. Si no fuese así, ver *id\_familia* no nulo indicaría que la categoría es helado. Además, justamente por esto, cuando se llevó el modelo a tablas en SQL, no se implementó una restricción sobre qué categorías pueden pertenecer a una familia. Si bien se va a empezar con Tortas Heladas, la empresa busca de a poco ir haciéndolo con todas las categorías con las que se pueda, y aún no define cuáles no podrían ser subdivididas.

### 3 Transformación al Modelo Relacional y Estructura de Tablas

#### 3.1 Modelo Relacional

| Relación $R(A_1, A_2, \dots)$                                                 | Clave primaria       | Claves candidatas      | Claves foráneas                                                                                   |
|-------------------------------------------------------------------------------|----------------------|------------------------|---------------------------------------------------------------------------------------------------|
| Categoría( <u>ID_cat</u> , nombre_cat )                                       | {ID_cat}             | {{ID_cat}}             | –                                                                                                 |
| Familia( <u>ID_fam</u> , nombre_fam )                                         | {ID_fam}             | {{ID_fam}}             | –                                                                                                 |
| Producto( <u>ID_prod</u> , nombre_prod, ID_cat, ID_fam )                      | {ID_prod}            | {{ID_prod}}            | {{ID_cat} ref Categoría;<br>{ID_fam} ref Familia}                                                 |
| Sucursal( <u>ID_suc</u> , nombre, calle, número, piso, depto, localidad, CP ) | {ID_suc}             | {{ID_suc}}             | –                                                                                                 |
| Usuario( <u>DNI</u> , nombre, fecha_nacimiento, mail, activo, ID_suc )        | {DNI}                | {{DNI}}                | {{ID_suc} ref Sucursal}                                                                           |
| Entrega( <u>ID_pedido</u> , DNI, DNI', fecha )                                | {ID_pedido}          | {{ID_pedido}}          | {{ID_pedido} ref Pedido;<br>{DNI} ref Proveedor;<br>{DNI'} ref Empleado;<br>{fecha} ref Recibe}   |
| Proveedor( <u>DNI</u> )                                                       | {DNI}                | {{DNI}}                | {{DNI} ref Usuario}                                                                               |
| Empleado( <u>DNI</u> , es_encargado )                                         | {DNI}                | {{DNI}}                | {{DNI} ref Usuario}                                                                               |
| Administrador( <u>DNI</u> )                                                   | {DNI}                | {{DNI}}                | {{DNI} ref Usuario}                                                                               |
| Pedido( <u>ID_pedido</u> , estado, ID_suc, <u>DNI</u> , DNI', fecha )         | {ID_pedido}          | {{ID_pedido}}          | {{ID_suc} ref Sucursal;<br>{DNI} ref Administrador;<br>{DNI'} ref Empleado;<br>{fecha} ref Emite} |
| Contiene( <u>ID_prod</u> , <u>ID_pedido</u> , cantidad )                      | {ID_prod, ID_pedido} | {{ID_prod, ID_pedido}} | {{ID_prod} ref Producto;<br>{ID_pedido} ref Pedido}                                               |

Cuadro 1: Modelo relacional: claves primarias, candidatas y foráneas.

En el modelo relacional se distinguen tres tipos de claves. La clave primaria es el atributo o conjunto de atributos que identifica de forma única a cada fila de una tabla, por ejemplo el DNI en la tabla de Usuario. Las claves candidatas son todos los posibles atributos que podrían cumplir ese rol de identificación, de entre ellas se elige una como clave primaria. Finalmente, las claves foráneas son atributos que establecen la relación entre dos tablas, ya que hacen referencia a la clave primaria de otra relación.

A partir del modelo Entidad-Relación se definieron las tablas del Modelo Relacional (Cuadro 1). Para cada entidad se generó una tabla con su clave primaria, mientras que las relaciones se transformaron en tablas adicionales sólo cuando fue necesario por la cardinalidad. Se implementó una tabla separada para cada subentidad (Proveedor, Empleado y Administrador) además de la del supertipo Usuario; estas subentidades heredan la clave primaria DNI.

La relación *Trabaja* entre Sucursal y Usuario es de tipo  $(1, N)$ , por lo que el identificador de sucursal ID\_suc se incluye como clave foránea en la tabla Usuario (el lado N contiene la FK).

Fue necesario modelar *Contiene*, relación entre Producto y Pedido, puesto que esta es  $(M, N)$  (un pedido puede incluir varios productos y un mismo producto puede formar parte de múltiples pedidos). Su clave primaria se formó por la combinación de ID\_pedido e ID\_prod. Posee su atributo *cantidad*, el cual indica cuánto de un producto fue pedido. En la tabla Producto se incluyeron no solo los atributos propios de la entidad, sino también las claves foráneas ID\_cat e ID\_fam, las cuales permiten incorporar las relaciones *Pertenece a* y *Parte de* directamente dentro de Producto, por ser ambas de tipo  $(1, N)$  Para las tablas Categoría y Familia se utilizaron simplemente sus atributos y claves primarias, sin necesitar de claves foráneas.

Para la entidad débil **Entrega**, se necesitó la clave foránea **ID\_pedido** proveniente de **Pedido**, dado que es la entidad fuerte de la cual depende, y este id resulta esencial para identificar qué se está entregando. Sus otras claves foráneas son **DNI ref Proveedor**, que viene de la relación *Provee* que tiene con **Proveedor** y es necesaria para conocer quién entregó el pedido, y **DNI ref Empleado** la cual viene de *Recibe*, y se necesita para saber quién controló el stock en el momento en el que llegó al local. También posee la fecha de cuando es realizada la entrega que viene de la relación *Recibe*.

Finalmente, en la entidad **Pedido** se utiliza su atributo **ID\_pedido** como clave primaria y posee un atributo estado, como fue explicado en la sección anterior. Se incluyen las claves foráneas **DNI ref Administrador**, de quien aprobó el encargo y **DNI ref Empleado**, quien lo emitió. Ambas vienen de las relaciones  $(1, N)$  *Aprueba* y *Emite* respectivamente. También de clave foránea tiene a **Id\_suc**, la cual viene de la relación *Corresponde*, y nos sirve para saber de qué sucursal es el pedido. Esta es muy importante, dado que, si un empleado cambia de sucursal y con anterioridad había hecho pedidos para otra sucursal, cambiar su **Id\_suc** en la base no afecta al pedido, porque este dato viene de la entidad **Sucursal** y no del usuario. Es importante aclarar que, en este modelado, interesa el recorrido de un pedido desde el momento que es generado hasta que es entregado. En este caso, no compete guardar en qué sucursales trabajó cada empleado, aunque esto de alguna manera podría inferirse si un empleado trabajó en dos sucursales diferentes y realizó pedidos en ambas. Si hubiese trabajado en alguna sucursal con anterioridad a donde trabaja actualmente, y no hubiese realizado un pedido para la misma, no podríamos saber si trabajó en esa sucursal antes o no. De todos modos, se reitera que este no es el objetivo de la base ni es relevante a la hora de guardar el recorrido de los pedidos.

### 3.2 Creación de la base y carga de datos

En esta sección describimos cómo llevamos los datos de los CSV al servidor PostgreSQL en Supabase y dejamos reproducibles los pasos para correr todo desde cero con un script que automatiza dicha carga llamado `manage_load.sh`.

**Infraestructura.** Creamos una base en Supabase y nos conectamos mediante una cadena de conexión guardada en un archivo `.env`.

```
SUPABASE_DB_URL=postgresql://<usuario>:<password>@<host>:<puerto>/<db>?sslmode=require
```

El código Python y los scripts SQL leen esta variable automáticamente (vía `python-dotenv`).

Con el fin de la verificación del modelo instanciado, se permite conectarse a la base sin necesidad de registrarse en Supabase y poder consultar los datos. Los datos de conexión son:

|                       |                                                  |
|-----------------------|--------------------------------------------------|
| <b>Host:</b>          | <code>aws-1-us-east-2.pooler.supabase.com</code> |
| <b>Puerto:</b>        | <code>6543</code>                                |
| <b>Base de datos:</b> | <code>postgres</code>                            |
| <b>Usuario:</b>       | <code>postgres.dejcrbhejidptuthxpy</code>        |
| <b>Contraseña:</b>    | <code>naoanianaathi</code>                       |
| <b>SSL:</b>           | <code>requerido</code>                           |

Siendo la cadena de conexión:

```
postgresql://postgres.dejcrbhejidptuthxpy:naoanianaathi@aws-1-us-east-2.pooler.supabase.com:6543/postgres
```

Para acceder a la base de datos es necesario instalar un cliente de PostgreSQL, como pgAdmin, DBeaver o bien utilizar la línea de comandos `psql`. En clientes gráficos como pgAdmin se debe crear un nuevo servidor (Add New Server → Connection) e ingresar los campos listados anteriormente. En la línea de comandos, en cambio, puede emplearse directamente la cadena de conexión mostrada. De esta manera, cualquier cliente compatible con PostgreSQL puede establecer la conexión y consultar las tablas con permisos de solo lectura.

Ahora, para cargar la base de datos se siguieron los pasos que vamos a detallar a continuación.

**De modelo lógico a físico.** Primero definimos el esquema físico en PostgreSQL que se muestra en la Fig. 3 a partir del Modelo Relacional discutido anteriormente (Cuadro 1).

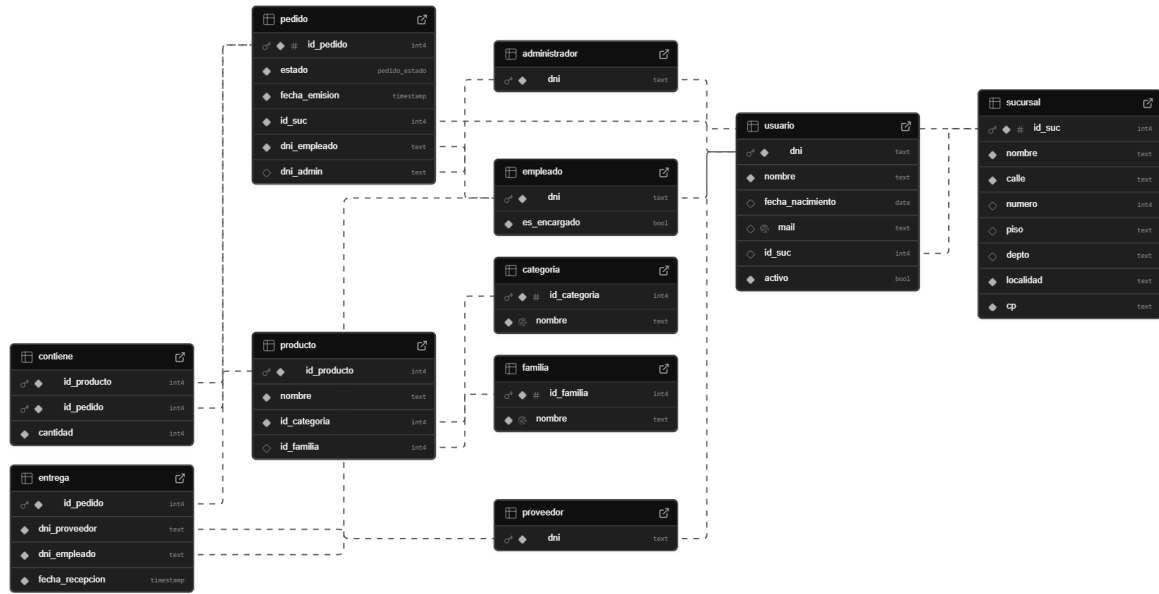


Figura 3: Modelo Relacional

En el archivo `sql/schema.sql` se define el esquema `app`, las tablas (`sucursal`, `usuario`, `empleado`, `administrador`, `proveedor`, `categoria`, `familia`, `producto`, `pedido`, `entrega`, `contiene`), los índices y la lógica de integridad (enums, *CHECKs* y *triggers*) que reflejan el flujo del negocio:

- **Flujo de Estados:** El Flujo de Estados del campo `pedido.estado`, que sigue las secuencias mostradas en el grafo de la figura 2, se garantiza principalmente mediante la función `app.chk_pedido_flujo_estado()` que se ejecuta con el disparador `trg_chk_pedido_flujo_estado` (BEFORE UPDATE). Esta función impone las reglas de transición: un pedido solo puede pasar a `aprobado` automáticamente si un administrador (`dni_admin`) agrega su DNI a un pedido ya emitido; solo puede cancelarse desde `emitido` y también con intervención administrativa, donde, no solo agrega su DNI, si no además aclara que quiere llevar el pedido al estado de cancelación. Se impide la cancelación en estados posteriores a (`aprobado`). El pedido debe estar `aprobado` antes de pasar a `preparado` o `entregado`.

No es necesario que un pedido haya pasado por el estado `preparado` para ser entregado, ya que el proveedor puede llevar en el camión productos extra. Para esta base de datos no interesa registrar exactamente el contenido del camión; si, mientras el camión está en circulación, se aprueba un pedido que coincide con esos productos adicionales, puede entregarse directamente. La progresión se completa con la función `app.auto_marcar_pedido_entregado()`, que utiliza el disparador `trg_auto_pedido_entregado` (AFTER INSERT OR UPDATE ON `app.entrega`) para actualizar el estado a `entregado` automáticamente una vez que se registra la recepción física de la mercancía. Por último, `entregado` es un estado final, lo que quiere decir que no puede modificarse una vez que el pedido ya fue entregado.

- **Restricciones de Roles y Ubicación y Lógica de Transición:**
  - **Emisión de un pedido:** La validación de la emisión del pedido se realiza a través de la función `app.pedido_validate_roles()`, ejecutada antes de la inserción (`trg_pedido_validate_roles_ins`) y antes de la actualización de campos clave (`trg_pedido_validate_roles_upd`) en la tabla `app.pedido`. La función chequea estrictamente el `DNI_empleado` para garantizar tres requisitos fundamentales de forma simultánea: **1)** Que el usuario sea un empleado (existe en `app.empleado`) y que tenga el rol de encargado (`es_encargado = TRUE`). **2)** Que el usuario se encuentre activo (`activo = TRUE` en `app.usuario`). **3)** Que el usuario pertenezca a la misma sucursal a la que se destina el pedido (`u.id_suc = NEW.id_suc`). Si cualquiera de estas condiciones falla, se tira una

excepción que impide la creación o modificación del pedido, manteniendo la integridad del proceso de emisión.

- **Roles:** Un registro en la tabla `app.usuario` puede existir sin estar asociado a ningún subtipo (`app.Empleado`, `app.administrador`, o `app.proveedor`). No se pueda registrar un administrador, empleado o proveedor si no existe previamente en la tabla `app.usuario`. Los roles de **Administrador** y **Proveedor** están restringidos exclusivamente a la sucursal ‘Casa Central’ (`trg_chk_admin_central` asegura que cualquier nuevo registro en la tabla `app.administrador` esté asociado a un usuario que pertenezca a la sucursal ‘Casa Central’, `trg_chk_proveedor_central` que realiza una validación del mismo tipo para `app.proveedor` y `chk_rolles_casa_central_usuario` la cual previene que un usuario con el rol de Administrador o Proveedor cambie su asignación a cualquier sucursal que no sea ‘Casa Central’).
- **Trazabilidad de Aprobación:** La trazabilidad de aprobación y cancelación que exige que el responsable (`DNI_admin`) sea un administrador activo se implementa mediante la función `app.pedido_validate_rolles()`, la cual es ejecutada antes de la inserción y de cualquier actualización de los campos responsables del pedido por los disparadores `trg_pedido_validate_rolles_ins` y `trg_pedido_validate_rolles_upd`. La función chequea el campo `DNI_admin` para asegurar que, si está presente, el usuario asociado: **1)** Tenga el rol de administrador (existe en `app.administrador`). **2)** Se encuentre activo (`activo = TRUE` en `app.usuario`).  
Si no se cumplen ambas condiciones, el sistema impide la operación. Adicionalmente, la función `app.chk_pedido_flujo_estado()` refuerza esta regla al obligar la presencia del administrador (`dni_admin` no nulo) para la transición de `emitido` a `aprobado` o `cancelado`.
- **Optimización Binaria:** La asignación del `DNI_admin` a un pedido en estado `emitido` fuerza **automáticamente** la transición al estado `aprobado`. Esta optimización se encuentra en la función `app.chk_pedido_flujo_estado()`.

#### ■ Coherencia Pedido–Entrega:

- **Receptor Ubicado:** El empleado que recibe la entrega (`app.entrega.DNI_empleado`) debe ser un **empleado activo** que pertenezca a la **misma sucursal** de destino del pedido. Se ejecuta mediante la función `app.chk_empleado_sucursal_entrega()`, que se activa antes de insertar o actualizar un registro en la tabla `app.entrega` por el disparador `trg_chk_empleado_sucursal_entrega`.
- **Finalización Automática:** Al registrar una entrega en `app.entrega` (inserción o actualización), el disparador `trg_auto_pedido_entregado` llama a la función `app.auto_marcar_pedido_entregado()`. Ésta actualiza el campo `estado` del pedido asociado a `entregado` (siempre que aún no lo esté), asegurando la coherencia entre el registro físico de la recepción y el estado lógico del pedido en la base. La operación se ejecuta dentro de la misma transacción que la inserción o actualización en la tabla `app.entrega`. Esto completa el flujo de estados sin requerir una intervención manual adicional.
- **Bloqueo por Cancelación:** No se puede insertar una entrega para un pedido que ya está `cancelado`. Se ejecuta mediante la función `app.chk_entrega_permitida()`, la cual es llamada antes de cualquier intento de insertar o actualizar un registro en la tabla `app.entrega` por el disparador `trg_chk_entrega_permitida`.

#### ■ Integridad de Producto y Cantidad:

- **Mínimo de Cantidad:** La cantidad de un producto solicitada en `app.contiene` debe ser **mayor que cero**. Se implementa de manera directa en `app.contiene` mediante la cláusula `CHECK (cantidad >0)`.
- **Contenido Mínimo:** Cada pedido debe contener al menos un producto. En este caso, la restricción se valida a través del propio diseño relacional de la tabla `app.contiene` sin necesidad de un trigger. Su clave primaria es compuesta por (`ID_producto`, `ID_pedido`) por ende se requiere la existencia de al menos un producto para registrar un pedido.

**Origen de los datos.** A partir de la heladería obtuvimos CSV's con productos, categorías, familias y pedidos. Esos archivos se encuentran en `csv_data/raw/`, los cuales tuvieron que ser preprocesados debido a la necesidad de normalizar encabezados (por ejemplo, separar categoría y familia, homogeneizar nombres de sucursales), y poder dejarlo en un formato limpio, dado que al venir de archivos excel habian columnas una debajo de la otra, columnas vacias, etc. Ese paso está automatizado en `converter_csv/paso_a_csv.py`. Este archivo `.py` está preparado para ejecutar el preprocesamiento completo o individual de archivos, según el modo seleccionado (*all* o *single*). Su funcionamiento detallado se describe en el `README.md` del proyecto. El resultado final queda en `csv_data/processed/`: un archivo por pedido (`pedidos_suc_<sucursal><n>.csv`) y los archivos de referencia `productos.csv`, `categorias.csv`, `familias.csv`.

También se utilizaron datos sintéticos, como los usuarios, debido a que la empresa decidió no proporcionar esta información para el trabajo práctico. Sin embargo, se generaron datos que representan con fidelidad cómo sería el formato de registro de usuarios en la vida real a partir de `generar_usuarios.py`. Estos se encuentran en `csv_data/fake/usuarios.csv`. En esa misma carpeta se encuentran CSV's individuales para administrados, empleados y proveedores. Estos muestran los DNI de los usuarios del tipo correspondiente que son correctos (por ejemplo, para el caso de administrador, aquellos de este tipo que su sucursal es Casa Central), por ende, los que están registrados en la base. En `usuarios.csv` existen dos usuarios incorrectos: un proveedor de DNI 45437166 y un administrador de DNI 46445321; los cuales no son registrados porque están asociados a una sucursal diferente a 'Casa Central', pero fueron puestos en el CSV para mostrar que, de querer cargarlos, se lanza la excepción y no se los carga en la base.

```
TP1-Bases_de_datos/scripts/cargar_usuarios.py
ERROR (Línea 72, DNI 46445321, Tipo: administrador): ERROR: Un administrador solo puede pertenecer a Casa Central (id_suc = 7). El usuario tiene ID_suc 8.
CONTEXT: PL/pgSQL function chk_admin_central() line 19 at RAISE
ERROR (Línea 73, DNI 45437166, Tipo: proveedor): ERROR: Un proveedor solo puede pertenecer a Casa Central (id_suc = 7). El usuario tiene ID_suc 8.
CONTEXT: PL/pgSQL function chk_proveedor_central() line 20 at RAISE

--- RESUMEN DE LA CARGA ---
✓ Registros procesados con éxito: 89
✗ Registros con error (Fila omitida y revertida): 2
La base de datos solo contiene los registros exitosos.
```

Figura 4: Carga de Usuarios omitiendo los incorrectos.

Para los datos de las sucursales, estos fueron directamente obtenidos desde internet, buscando a cada sucursal y anotándolos en `sucursales.sql`

**Carga incremental (scripts).** Para insertar los datos en las tablas seguimos este orden, desde `scripts/`. Cada script usa la variable de conexión `SUPABASE_DB_URL`:

1. `cargar_categorias.py`: lee `csv_data/categorias.csv` y sube `app.categoria`.
2. `cargar_familias.py`: lee `csv_data/familias.csv` y sube `app.familia`.
3. `cargar_productos.py`: lee `csv_data/productos.csv` y sube `app.producto` enlazando con categoría/familia.
4. `cargar_sucursales.py`: crea y carga `app.sucursal` a partir de `sucursales.sql`.
5. `cargar_usuarios.py`: Carga usuario, empleado, administrador y proveedor por sucursal.
6. `cargar_pedidos.py`: este script es responsable de transformar cada archivo procesado en `csv_data/processed/` en un registro de pedido (`app.pedido`) y sus líneas asociadas (`app.contiene`). El comportamiento del script contempla dos modos operativos seleccionables por parámetro:
  - **emit\_all**: modo ideado para poblar la base con pedidos iniciales. Crea todos los pedidos en estado emitido, no asigna `DNI_admin` ni genera registros de `app.entrega`, y solo inserta las líneas en `app.contiene`. Es útil cuando se quiere dejar la base en un estado inicial.
  - **random** (por defecto): pensamos un modo para simular un flujo operativo realista. Asigna automáticamente un empleado encargado de la sucursal correspondiente, decide (según las reglas explicadas anteriormente) cuándo asignar un `DNI_admin` para forzar la aprobación o la cancelación y, cuando corresponde, inserta filas en `app.entrega` para marcar entregas (estos inserts activan los triggers del esquema que sincronizan el estado del pedido). Este modo se utiliza para obtener datos que prueben todas las rutas del negocio: emisión, aprobación, cancelación y entrega.

En ambos modos, el script carga la fecha de emisión a partir de la columna del CSV del pedido que lleva el nombre ‘fecha’. Por cómo se preprocesan los datos, si el archivo Excel recibido no cuenta con ninguna, dicha columna del CSV preprocesado se conformará de NULLs. En ese caso, este script asignará automáticamente la fecha y hora del momento en el que se ejecuta. En lo que tiene que ver con los pedidos entregados, esta fecha siempre será la del momento en el que se efectúa dicha entrega, ya sea a partir de este script (con el modo `random`) o a partir de las operaciones SQL que describiremos en la subsección siguiente.

Para la ejecución y parámetros exactos se remite al `README.md` del repositorio.

La carga, tanto del `schema` como de los datos, está automatizada en un archivo llamado `manage_load.sh`, que concentra todo el flujo descrito, y se puede ver más información sobre los parámetros de ejecución en el `README.md`.

**Estados y entregas.** Como dijimos anteriormente, en la carga de datos es posible dejar la base en un estado inicial (modo `emit_all`) desde el cual se pueden probar manualmente las transiciones mediante consultas SQL; alternatively, en modo `random` el script ya aplica decisiones automáticas (aprobación/cancelación/entrega) pero también se puede operar sobre esta disposición. Para recorrer el flujo desde un pedido emitido se pueden usar operaciones SQL simples, por ejemplo:

-- Aprobar (asignar admin; el trigger convierte la asignación en ‘aprobado’)

```
1 UPDATE app.pedido SET dni_admin = '51105439'
2 WHERE id_pedido = 8 AND estado = 'emitido';
```

-- Pasar a preparado (válido solo desde ‘aprobado’)

```
1 UPDATE app.pedido SET estado = 'preparado'
2 WHERE id_pedido = 3 AND estado = 'aprobado';
```

-- Registrar entrega (insertar en app.entrega y el trigger marca automáticamente el pedido como ‘entregado’)

```
1 INSERT INTO app.entrega (id_pedido, dni_proveedor, dni_empleado)
2 VALUES (5, '82647125', '11946177');
```

Los triggers del esquema que explicamos anteriormente implementan las validaciones críticas (roles, sucursal, secuencia de estados y coherencia pedido–entrega), por lo que las consultas anteriores van a ser rechazadas si no cumplen dichas precondiciones.

## 4 Consultas SQL

Para ilustrar el uso de SQL sobre nuestro modelo relacional, diseñamos una serie de consultas que combinan operaciones esenciales del lenguaje como `JOIN`, `WHERE`, `HAVING`, `COUNT`, `AVG` y `LIMIT`. Cada query fue pensada para reflejar necesidades reales del negocio de la heladería: desde obtener listas básicas de productos o usuarios hasta detectar situaciones más complejas como pedidos no entregados o promedios de operaciones. En el gestor que utilizamos, todas las tablas (administrador, categoria, contiene, etc.) residen en el esquema llamado `app`. Por lo tanto, para ejecutar las consultas de forma correcta, es necesario especificar el nombre del esquema antes del nombre de la tabla (ej. `app.tabla`).

**1) Empleados por sucursal.** Obtiene la cantidad de empleados por sucursal mediante `JOIN` entre `Sucursal`, `Usuario` y `Empleado`. Se agrupa por sucursal y se ordena de mayor a menor con `ORDER BY` para identificar las sedes con más personal.

```
1 SELECT s.ID_suc, s.nombre AS sucursal, COUNT(e.DNI) AS cant_empleados
2 FROM app.Sucursal s
3 JOIN app.Usuario u ON u.ID_suc = s.ID_suc
4 JOIN app.Empleado e ON e.DNI = u.DNI
5 GROUP BY s.ID_suc, s.nombre
6 ORDER BY cant_empleados DESC;
```

Resultado:

| id_suc | sucursal        | cant_empleados |
|--------|-----------------|----------------|
| 2      | Catamarca       | 11             |
| 4      | Refinor         | 11             |
| 5      | Lobo de la Vega | 11             |
| 3      | Barrio Norte    | 10             |
| 6      | Quara           | 10             |
| 7      | Casa Central    | 10             |
| 1      | Microcentro     | 9              |
| 8      | Tafi            | 9              |

**2) Top 3 empleados que más pedidos emitieron.** Relaciona Empleado con Pedido, cuenta pedidos por persona y muestra los tres más activos usando ORDER BY y LIMIT.

```

1 SELECT u.DNI, u.nombre, COUNT(p.ID_pedido) AS pedidos_emitidos
2 FROM app.Empleado e
3 JOIN app.Usuario u ON u.DNI = e.DNI
4 JOIN app.Pedido p ON p.DNI_EMPLEADO = e.DNI
5 GROUP BY u.DNI, u.nombre
6 ORDER BY pedidos_emitidos DESC
7 LIMIT 3;

```

Resultado:

| dni      | nombre                  | pedidos_emitidos |
|----------|-------------------------|------------------|
| 15015854 | Natalio Gallego Tello   | 5                |
| 79377751 | Reinaldo Roldan Palmer  | 4                |
| 79785602 | Eloísa Salamanca Melero | 4                |

**3) Pedidos por estado y sucursal en el mes de Septiembre de 2025.** Vincula cada pedido con la sucursal del empleado que lo emitió, filtra por período con WHERE, y resume por sucursal y estado mediante GROUP BY.

```

1 SELECT s.nombre AS sucursal, p.estado, COUNT(*) AS cant
2 FROM app.pedido p
3 JOIN app.usuario u ON u.DNI = p.DNI_EMPLEADO
4 JOIN app.sucursal s ON s.ID_suc = u.ID_suc
5 WHERE CAST(p.fecha_emision AS DATE) BETWEEN '2025-09-01' AND '2025-09-30'
6 GROUP BY s.nombre, p.estado
7 ORDER BY s.nombre, p.estado;

```

Resultado:

| sucursal     | estado    | cant | sucursal        | estado    | cant |
|--------------|-----------|------|-----------------|-----------|------|
| Barrio Norte | emitido   | 1    | Lobo de la Vega | emitido   | 2    |
| Barrio Norte | aprobado  | 1    | Lobo de la Vega | preparado | 1    |
| Barrio Norte | preparado | 1    | Lobo de la Vega | entregado | 1    |
| Barrio Norte | entregado | 1    | Microcentro     | emitido   | 1    |
| Catamarca    | aprobado  | 2    | Microcentro     | preparado | 1    |
| Catamarca    | preparado | 1    | Microcentro     | entregado | 1    |
| Catamarca    | cancelado | 1    |                 |           |      |

**4) Pedidos que aún no tuvieron entrega o no van a tener (cancelados) (anti-join).** Detecta pedidos sin ninguna entrega asociada utilizando un LEFT JOIN entre Pedido y Entrega y filtrando con WHERE e.ID\_pedido IS NULL. Esta consulta es útil para auditoría y control operativo, ya que identifica solicitudes emitidas pero no satisfechas, pudiendo además ordenarse por fecha para priorizar las más recientes.

```

1 SELECT p.id_pedido, p.estado, p.fecha_emision, u.nombre AS empleado
2 FROM app.pedido p
3 JOIN app.usuario u ON u.DNI = p.DNI_EMPLEADO
4 LEFT JOIN app.entrega e ON e.ID_pedido = p.ID_pedido
5 WHERE e.id_pedido IS NULL
6 ORDER BY p.fecha_emision DESC;

```

Resultado:

| id_pedido | estado    | fecha_emision | empleado                |
|-----------|-----------|---------------|-------------------------|
| 16        | emitido   | 2025-09-15    | Rocío Llorente Atienza  |
| 5         | cancelado | 2025-09-15    | Natalio Gallego Tello   |
| 9         | emitido   | 2025-09-14    | Reinaldo Roldan Palmer  |
| 12        | emitido   | 2025-09-14    | Eloísa Salamanca Melero |
| 4         | aprobado  | 2025-09-14    | Natalio Gallego Tello   |
| 8         | aprobado  | 2025-09-14    | Reinaldo Roldan Palmer  |
| 15        | emitido   | 2025-09-14    | Rocío Llorente Atienza  |
| 11        | preparado | 2025-09-10    | Eloísa Salamanca Melero |
| 3         | aprobado  | 2025-09-10    | Natalio Gallego Tello   |
| 7         | preparado | 2025-09-10    | Reinaldo Roldan Palmer  |
| 6         | preparado | 2025-09-10    | Rocío Llorente Atienza  |
| 2         | preparado | 2025-09-08    | Natalio Gallego Tello   |

**5) Productos más pedidos (suma de cantidades).** Suma las cantidades pedidas por producto con SUM y muestra los más populares. Usa la columna cantidad en Contiene.

```

1 SELECT pr.id_producto, pr.nombre, SUM(c.cantidad) AS total_unidades
2 FROM app.contiene c
3 JOIN app.producto pr ON pr.id_producto = c.id_producto
4 GROUP BY pr.id_producto, pr.nombre
5 ORDER BY total_unidades DESC
6 LIMIT 10;

```

Resultado:

| id_producto | nombre                   | total_unidades |
|-------------|--------------------------|----------------|
| 740         | DULCE DE LECHE           | 57             |
| 723         | CHOCOLATE                | 55             |
| 764         | TIRAMISU                 | 47             |
| 710         | BOMBON ROCHER            | 28             |
| 782         | CHOCOLATE DUBAI          | 26             |
| 732         | DELICIA TELLO            | 25             |
| 749         | FRUTILLA A LA CREMA      | 25             |
| 730         | CHOCOLATE SUIZO          | 24             |
| 743         | DULCE DE LECHE GRANIZADO | 23             |
| 706         | ARANDANO Y CASSIS        | 22             |

**6) Tiempo promedio de entrega de una sucursal.** Calcula el tiempo promedio en horas que pasa entre la emisión de un pedido y su recepción. La consulta une las tablas **Pedido**, **Entrega** y **Sucursal** y utiliza la función **EXTRACT** para obtener la diferencia en segundos entre las fechas. Luego, divide por 3600 (segundos por hora) para obtener el resultado en horas y lo redondea a 2 decimales. Finalmente, agrupa por sucursal para obtener el promedio de cada una.

```
1 SELECT s.nombre AS sucursal,
2        ROUND(AVG(EXTRACT(EPOCH FROM (e.fecha_recepcion - p.fecha_emision)) / 3600), 2) AS
   horas_promedio
3 FROM app.pedido p
4 JOIN app.entrega e ON e.id_pedido = p.id_pedido
5 JOIN app.sucursal s ON s.id_suc = p.id_suc
6 GROUP BY s.nombre
7 ORDER BY horas_promedio;
```

Resultado:

| sucursal        | horas_promedio |
|-----------------|----------------|
| Barrio Norte    | 961.42         |
| Microcentro     | 961.43         |
| Lobo de la Vega | 961.45         |
| Catamarca       | 1465.40        |

*Recordar que si bien los fecha de emisión es un dato real la de entrega no, y tiene que ver con cuándo hicimos las entregas nosotras ¡por eso dan números tan grandes!*

**7) Categoría más pedida del 2025.** Relaciona Contiene, Producto, Categoría y Pedido, y filtra los pedidos realizados en el año 2025 con **WHERE**. Agrupa los resultados por categoría y suma las cantidades de productos pedidos, ordenando el resultado de mayor a menor y limitando la salida a la primera fila con **LIMIT 1** para mostrar solo la categoría más popular.

```
1 SELECT c.nombre AS categoria, SUM(co.cantidad) AS total_pedido
2 FROM app.contiene co
3 JOIN app.producto pr ON pr.id_producto = co.id_producto
4 JOIN app.categoria c ON c.id_categoria = pr.id_categoria
5 JOIN app.pedido p ON p.id_pedido = co.id_pedido
6 WHERE EXTRACT(YEAR FROM p.fecha_emision) = 2025
7 GROUP BY c.nombre
8 ORDER BY total_pedido DESC
9 LIMIT 1;
```

Resultado:

| categoria | total_pedido |
|-----------|--------------|
| helado    | 830          |

**8) Proveedores que más pedidos entregaron con Banana Split.** Busca a los proveedores que hayan entregado la mayor cantidad de pedidos que contenían el producto "Banana Split". Se unen las tablas **Entrega**, **Proveedor**, **Usuario**, **Contiene** y **Producto**. La cláusula **WHERE** filtra los resultados para incluir solo los pedidos que contienen el producto buscado. Finalmente, se agrupa por proveedor para contar los pedidos únicos entregados por cada uno.

```
1 SELECT pr.dni AS proveedor, u.nombre, COUNT(DISTINCT e.id_pedido) AS pedidos_con_banana_split
2 FROM app.entrega e
3 JOIN app.proveedor pr ON pr.dni = e.dni_proveedor
4 JOIN app.usuario u ON u.dni = pr.dni
5 JOIN app.contiene co ON co.id_pedido = e.id_pedido
6 JOIN app.producto p ON p.id_producto = co.id_producto
7 WHERE p.nombre ILIKE 'BANANA SPLIT'
8 GROUP BY pr.dni, u.nombre
9 ORDER BY pedidos_con_banana_split DESC;
```

Resultado:

| proveedor | nombre                             | pedidos_con_banana_split |
|-----------|------------------------------------|--------------------------|
| 37146170  | Heliodoro Cecilio Bermejo Rocamora | 4                        |

**9) Total unidades pedidas por familia.** La consulta vincula las tablas *Contiene*, *Producto* y *Familia*. Agrupa los resultados por familia y utiliza la función *SUM* sobre la columna *cantidad* de la tabla *Contiene* para obtener la cantidad total de unidades pedidas de cada familia.

```

1 SELECT f.nombre AS familia, SUM(c.cantidad) AS total_pedido
2 FROM app.contiene c
3 JOIN app.producto pr ON pr.id_producto = c.id_producto
4 JOIN app.familia f ON f.id_familia = pr.id_familia
5 GROUP BY f.nombre
6 ORDER BY total_pedido DESC;
```

Resultado:

| familia        | total_pedido |
|----------------|--------------|
| crema blanca   | 376          |
| chocolate      | 205          |
| dulce de leche | 159          |
| crema vainilla | 54           |
| frutas         | 36           |

**10) Recuento de productos por categoría.** Se unen las tablas *Categoría* y *Producto*, agrupando los resultados por el nombre de la categoría. Se usa *COUNT* para obtener el recuento de productos para cada una.

```

1 SELECT c.nombre AS categoria, COUNT(p.id_producto) AS cant_productos
2 FROM app.categoria c
3 JOIN app.producto p ON p.id_categoria = c.id_categoria
4 GROUP BY c.nombre
5 ORDER BY cant_productos DESC;
```

Resultado:

| categoria      | cant_productos |
|----------------|----------------|
| helado         | 56             |
| Paletas        | 14             |
| Tortas Heladas | 9              |
| Tartas         | 9              |
| diet           | 6              |
| Tortas         | 6              |
| Minitortas     | 3              |

**11) Administrador que más pedidos aceptó.** Esta consulta cuenta el número de productos existentes en cada categoría. Vincula las tablas *categoría* y *producto*, agrupa los resultados por el nombre de la categoría y usa la función *COUNT* para obtener el total de productos.

```

1 SELECT
2   u.nombre AS nombre_administrador,
3   COUNT(p.id_pedido) AS total_pedidos_aceptados
4 FROM
5   app.administrador AS a
6   JOIN app.usuario AS u
```

```
7      ON a.dni = u.dni
8      JOIN app.pedido AS p
9      ON a.dni = p.dni_admin
10     GROUP BY
11         u.nombre
12     ORDER BY
13         total_pedidos_aceptados DESC
14     LIMIT 1;
```

Resultado:

| nombre_administrador | total_pedidos_aceptados |
|----------------------|-------------------------|
| Adrián de Diaz       | 11                      |

## 5 Conclusiones y Uso Futuro

El trabajo permitió analizar un problema de la vida real, estudiarlo y proponer una solución que optimiza recursos y tiempo. Se recorrió paso a paso el trayecto de un pedido, desde que se emite hasta que se entrega, incorporando una medida de seguridad fundamental que es registrar quiénes son responsables de la mercadería en tránsito. Esta decisión resulta crucial porque hasta el momento no estaba resuelta de manera adecuada.

Se analizó exhaustivamente la construcción del modelo E-R explorando diferentes variantes y decidiendo finalmente por aquella que mejor representa el proceso real que se buscaba modelar. Al momento de trasladar el diseño al modelo relacional y a la base en SQL surgieron nuevas problemáticas vinculadas con permisos, relaciones y valores nulos, las cuales fueron abordadas mediante distintas pruebas hasta lograr un esquema fiel a las necesidades de la empresa. No solo se analizó el funcionamiento actual, sino que también se contemplaron posibles cambios futuros de la empresa, modelando la base de datos de manera que su adaptación resulte sencilla y escalable.

Actualmente se está desarrollando la web mediante la cual esta base de datos será utilizada a diario, incorporando algunos cambios mínimos que resultan más adecuados en la práctica. En esta se van a encontrar registrados los distintos tipos de usuarios con accesos diferenciados donde el empleado encargado puede realizar pedidos para la sucursal en la que trabaja, el administrador tiene la posibilidad de visualizarlos, aprobarlos o rechazarlos y el proveedor únicamente puede consultarlos para efectuar la entrega de productos. De esta manera se garantiza la seguridad de la información evitando que se modifiquen datos de manera indebida como sucede hoy en día con el uso de Excel.

La división de los tipos de usuarios planteada en este trabajo y ya se encuentra implementada en la plataforma disponible en <https://reportes-produccion.onrender.com/>. Los próximos pasos consisten en diseñar interfaces más amigables y establecer la conexión con la base de datos en Supabase de modo que los pedidos se registren y actualicen automáticamente con simples interacciones del usuario.

## 6 Referencias

- [1] IONOS. “¿Qué es una base de datos? Funciones y modelos”, \*Digital Guide – IONOS España\*, 6 de marzo de 2024. Disponible en: <https://ionos.es/digitalguide/hosting/cuestiones-tecnicas/bases-de-datos/> [Consultado: 18 de Septiembre de 2025].